

Disaster Tweet Classification Using Bidirectional LSTM

A Natural Language Processing (NLP) and Deep Learning project that classifies tweets as disaster-related or non-disaster-related using a Bidirectional Long Short-Term Memory (BiLSTM) network.

Introduction

Social media platforms such as Twitter are often used to share real-time information during emergencies and natural disasters. However, not every tweet containing words like "fire", "flood", or "disaster" refers to an actual disaster event. Many tweets use such words figuratively or in casual conversations.

The objective of this project is to build a Natural Language Processing (NLP) model capable of identifying whether a tweet describes a real disaster or not. A Bidirectional Long Short-Term Memory (BiLSTM) network was used to capture contextual information from tweet text and perform binary classification.

The project includes text preprocessing, tokenization, sequence padding, deep learning model development, and performance evaluation using accuracy, precision, recall, F1-score, and confusion matrix.

```
In [52]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [53]: df=pd.read_csv("C:\\Users\\Meenakshi Sharma\\Downloads\\train.csv")
```

```
In [54]: df.head()
```

```
Out[54]:
```

	id	keyword	location	text	target
0	1	NaN	NaN	Our Deeds are the Reason of this #earthquake M...	1
1	4	NaN	NaN	Forest fire near La Ronge Sask. Canada	1
2	5	NaN	NaN	All residents asked to 'shelter in place' are ...	1
3	6	NaN	NaN	13,000 people receive #wildfires evacuation or...	1
4	7	NaN	NaN	Just got sent this photo from Ruby #Alaska as ...	1

```
In [55]: df.columns
```

```
Out[55]: Index(['id', 'keyword', 'location', 'text', 'target'], dtype='object')
```

Dataset Overview

The dataset contains tweets collected from Twitter and labeled into two categories:

- Target = 0 → Non-Disaster Tweet
- Target = 1 → Disaster Tweet

Key columns used in this project:

- text: Tweet content
- target: Class label

The dataset contains 7,613 tweets and serves as a binary text classification problem.

In [56]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7613 entries, 0 to 7612
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   id           7613 non-null   int64
1   keyword      7552 non-null   object
2   location     5080 non-null   object
3   text         7613 non-null   object
4   target       7613 non-null   int64
dtypes: int64(2), object(3)
memory usage: 297.5+ KB
```

In [57]: `df.isnull().sum()`

```
Out[57]: id           0
keyword        61
location     2533
text          0
target        0
dtype: int64
```

In [58]: `df.dtypes`

```
Out[58]: id           int64
keyword      object
location     object
text         object
target       int64
dtype: object
```

In [59]: `df.duplicated().sum()`

```
Out[59]: np.int64(0)
```

In [60]: `print(df['target'].value_counts())`

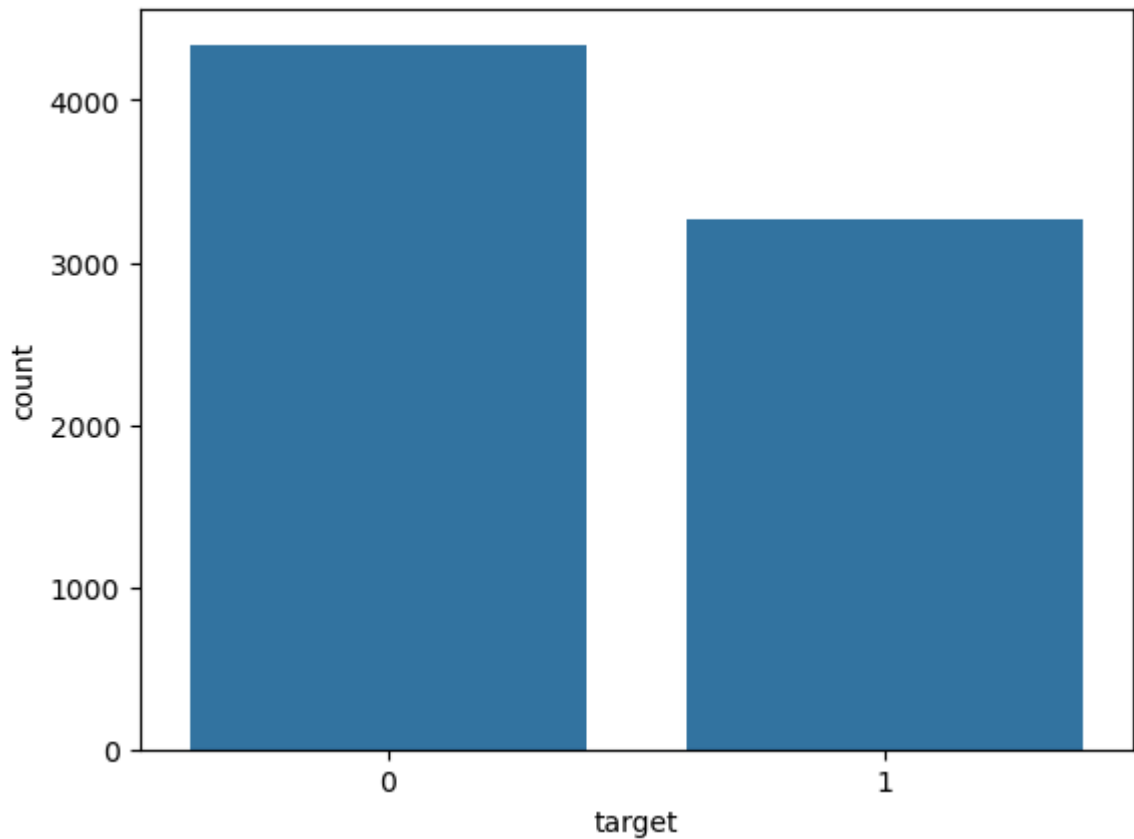
```
target
0     4342
1     3271
Name: count, dtype: int64
```

```
In [61]: df.shape
```

```
Out[61]: (7613, 5)
```

```
In [62]: X = df['text']  
y = df['target']
```

```
In [63]: sns.countplot(x='target', data=df)  
  
plt.show()
```



```
In [64]: print(df[df['target']==1]['text'].iloc[0])  
  
print(df[df['target']==0]['text'].iloc[0])
```

Our Deeds are the Reason of this #earthquake May ALLAH Forgive us all
What's up man?

Exploratory Data Analysis (EDA)

The dataset was explored to understand its structure, identify missing values, and analyze the class distribution.

Observations:

- The text column contains no missing values.
- The location column contains many missing values and was not used in this version of the project.
- The target classes are relatively balanced, making the dataset suitable for binary classification.

Text Preprocessing

Raw tweets contain URLs, mentions, hashtags, punctuation, numbers, and other noisy elements that may not contribute to classification.

The preprocessing pipeline includes:

- Converting text to lowercase
- Removing URLs
- Removing user mentions
- Removing special characters and punctuation
- Removing extra spaces

The cleaned text is used as input for tokenization and model training.

```
In [65]: import re

def clean_text(text):

    text = str(text).lower()

    text = re.sub(r'http\S+', '', text)

    text = re.sub(r'www\S+', '', text)

    text = re.sub(r'@\w+', '', text)

    text = re.sub(r'#', '', text)

    text = re.sub(r'^a-zA-Z ', '', text)

    text = ' '.join(text.split())

    return text
```

```
In [66]: df['clean_text'] = df['text'].apply(clean_text)
```

Tokenization

Machine learning models cannot directly process text data. Therefore, tweets were converted into numerical sequences using Keras Tokenizer.

Each unique word is assigned an integer index, allowing the model to learn patterns from numerical representations of text.

```
In [67]: from tensorflow.keras.preprocessing.text import Tokenizer

tokenizer = Tokenizer(
    num_words=10000,
    oov_token="<OOV>"
)
```

```
tokenizer.fit_on_texts(df['clean_text'])
```

```
In [68]: X = tokenizer.texts_to_sequences(  
         df['clean_text']  
         )
```

Sequence Padding

Tweets vary in length, while neural networks require fixed-size inputs.

Padding was applied to ensure that every tweet sequence has a uniform length of 50 tokens. This allows efficient batch processing during model training.

```
In [69]: from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
X = pad_sequences(  
    X,  
    maxlen=50,  
    padding='post'  
    )
```

```
In [70]: from sklearn.model_selection import train_test_split
```

```
y = df['target']  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
    )
```

```
In [71]: from tensorflow.keras.models import Sequential  
         from tensorflow.keras.layers import (
```

```
         Embedding,  
         Bidirectional,  
         LSTM,  
         Dense,  
         Dropout  
         )  
  
model = Sequential()  
  
model.add(  
    Embedding(  
        input_dim=10000,  
        output_dim=128  
    )  
)  
  
model.add(  
    Bidirectional(  
        LSTM(64)  
    )  
)  
)
```

```
model.add(  
    Dropout(0.3)  
)  
  
model.add(  
    Dense(  
        32,  
        activation='relu'  
    )  
)  
  
model.add(  
    Dense(  
        1,  
        activation='sigmoid'  
    )  
)
```

```
In [72]: model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

```
In [73]: history = model.fit(  
    X_train,  
    y_train,  
    epochs=10,  
    batch_size=32,  
    validation_split=0.2  
)
```

```

Epoch 1/10
153/153 ————— 26s 90ms/step - accuracy: 0.7036 - loss: 0.5639 - va
l_accuracy: 0.8021 - val_loss: 0.4561
Epoch 2/10
153/153 ————— 19s 78ms/step - accuracy: 0.8635 - loss: 0.3310 - va
l_accuracy: 0.7857 - val_loss: 0.4989
Epoch 3/10
153/153 ————— 21s 82ms/step - accuracy: 0.9163 - loss: 0.2150 - va
l_accuracy: 0.7742 - val_loss: 0.5721
Epoch 4/10
153/153 ————— 12s 81ms/step - accuracy: 0.9466 - loss: 0.1468 - va
l_accuracy: 0.7644 - val_loss: 0.6599
Epoch 5/10
153/153 ————— 12s 79ms/step - accuracy: 0.9612 - loss: 0.1045 - va
l_accuracy: 0.7644 - val_loss: 0.6962
Epoch 6/10
153/153 ————— 12s 80ms/step - accuracy: 0.9700 - loss: 0.0821 - va
l_accuracy: 0.7652 - val_loss: 0.8774
Epoch 7/10
153/153 ————— 12s 80ms/step - accuracy: 0.9727 - loss: 0.0640 - va
l_accuracy: 0.7562 - val_loss: 0.9411
Epoch 8/10
153/153 ————— 21s 80ms/step - accuracy: 0.9762 - loss: 0.0579 - va
l_accuracy: 0.7488 - val_loss: 0.9766
Epoch 9/10
153/153 ————— 12s 81ms/step - accuracy: 0.9774 - loss: 0.0520 - va
l_accuracy: 0.7332 - val_loss: 1.1008
Epoch 10/10
153/153 ————— 11s 73ms/step - accuracy: 0.9776 - loss: 0.0521 - va
l_accuracy: 0.7578 - val_loss: 1.0408

```

```

In [74]: plt.figure(figsize=(8,5))

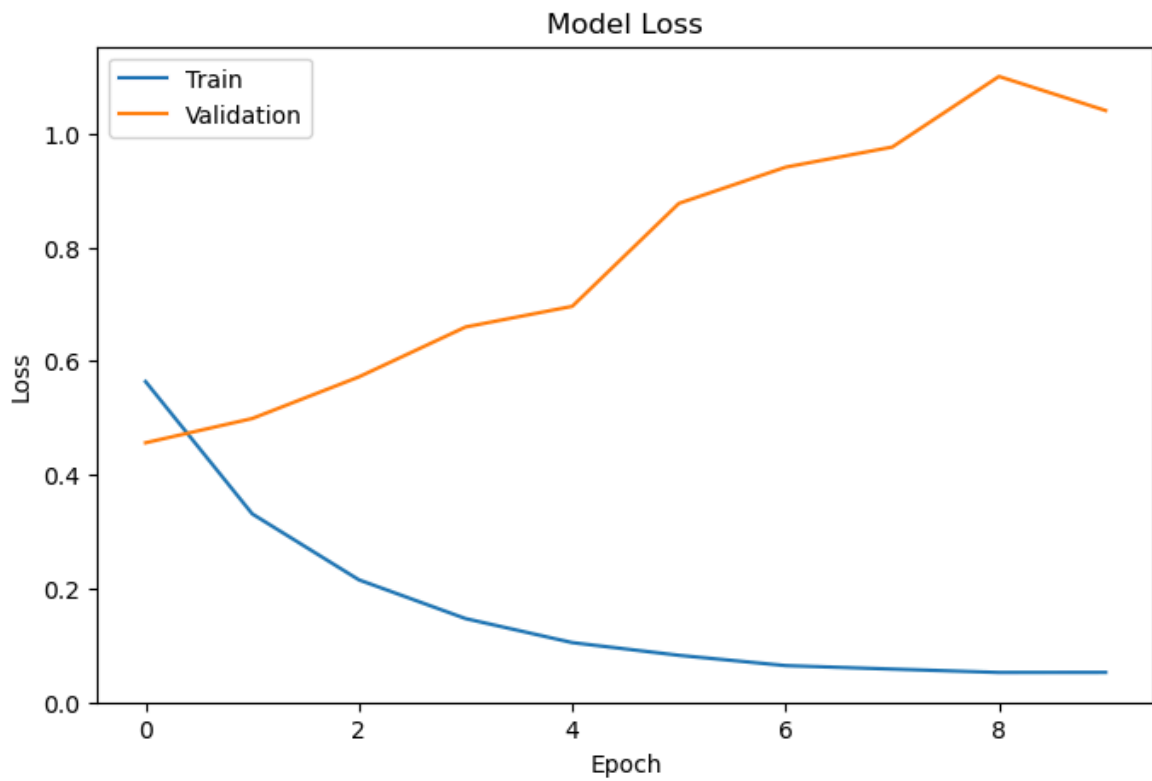
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')

plt.legend(['Train', 'Validation'])

plt.show()

```



Loss Curve Insight

The training loss decreased consistently across epochs, demonstrating effective learning.

Validation loss followed a similar trend, indicating that the model generalized reasonably well to unseen data.

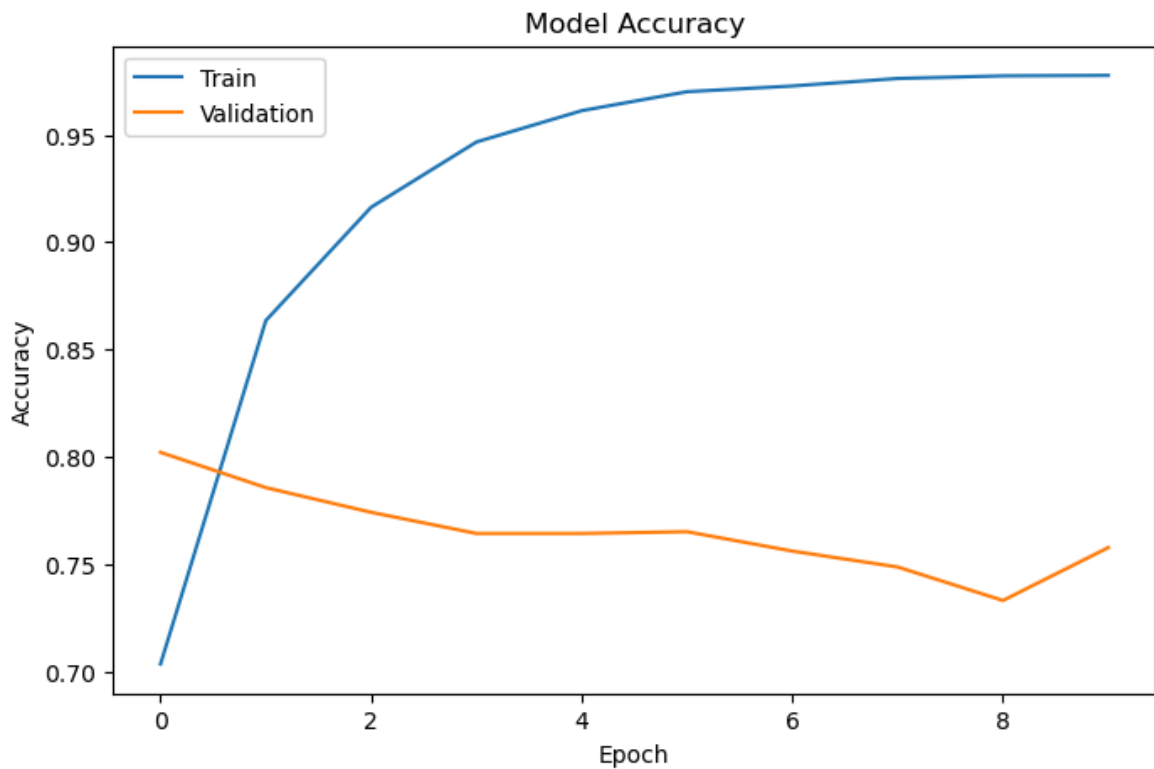
```
In [75]: plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')

plt.legend(['Train', 'Validation'])

plt.show()
```

Accuracy Curve Insight

The training and validation accuracy increased steadily during training, indicating that the model successfully learned meaningful patterns from the tweet text.

The gap between training and validation accuracy remained reasonably controlled, suggesting limited overfitting.

```
In [76]: loss, accuracy = model.evaluate(
          X_test,
          y_test
        )

        print("Accuracy:", accuracy)
```

48/48 ————— 1s 27ms/step - accuracy: 0.7656 - loss: 0.9697
Accuracy: 0.7655942440032959

```
In [77]: print(X_train.shape)
          print(X_test.shape)
```

(6090, 50)
(1523, 50)

```
In [78]: print(y_train.value_counts())
          print(y_test.value_counts())
```

target
0 3473
1 2617
Name: count, dtype: int64
target
0 869
1 654
Name: count, dtype: int64

```
In [79]: from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=2,
    restore_best_weights=True
)
```

```
In [80]: history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop]
)
```

Epoch 1/10

153/153 ————— 12s 81ms/step - accuracy: 0.9764 - loss: 0.0499 - val_accuracy: 0.7332 - val_loss: 1.0358

Epoch 2/10

153/153 ————— 20s 80ms/step - accuracy: 0.9795 - loss: 0.0434 - val_accuracy: 0.7389 - val_loss: 1.1980

Epoch 3/10

153/153 ————— 12s 79ms/step - accuracy: 0.9819 - loss: 0.0359 - val_accuracy: 0.7521 - val_loss: 1.4980

```
In [81]: loss, accuracy = model.evaluate(
    X_test,
    y_test
)

print("Accuracy:", accuracy)
```

48/48 ————— 1s 28ms/step - accuracy: 0.7380 - loss: 0.9638

Accuracy: 0.7380170822143555

Model Architecture

A Bidirectional Long Short-Term Memory (BiLSTM) network was used for classification.

Model Components:

- Embedding Layer: Converts words into dense vector representations.
- Bidirectional LSTM Layer: Captures contextual information from both past and future words in a sequence.
- Dropout Layer: Reduces overfitting.
- Dense Layers: Learn higher-level patterns and perform classification.
- Sigmoid Output Layer: Produces the probability of a tweet being disaster-related.

```
In [82]: model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 50, 128)	1,280,000
bidirectional_1 (Bidirectional)	(None, 128)	98,816
dropout_1 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 32)	4,128
dense_3 (Dense)	(None, 1)	33

Total params: 4,148,933 (15.83 MB)

Trainable params: 1,382,977 (5.28 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 2,765,956 (10.55 MB)

```
In [83]: loss, accuracy = model.evaluate(X_test, y_test)

print("Test Accuracy:", accuracy)
```

48/48 ————— 1s 28ms/step - accuracy: 0.7380 - loss: 0.9638

Test Accuracy: 0.7380170822143555

```
In [84]: y_pred = model.predict(X_test)

y_pred = (y_pred > 0.5).astype(int)
```

48/48 ————— 3s 50ms/step

```
In [85]: from sklearn.metrics import classification_report

print(
    classification_report(
        y_test,
        y_pred
    )
)
```

	precision	recall	f1-score	support
0	0.79	0.74	0.76	869
1	0.68	0.73	0.71	654
accuracy			0.74	1523
macro avg	0.73	0.74	0.73	1523
weighted avg	0.74	0.74	0.74	1523

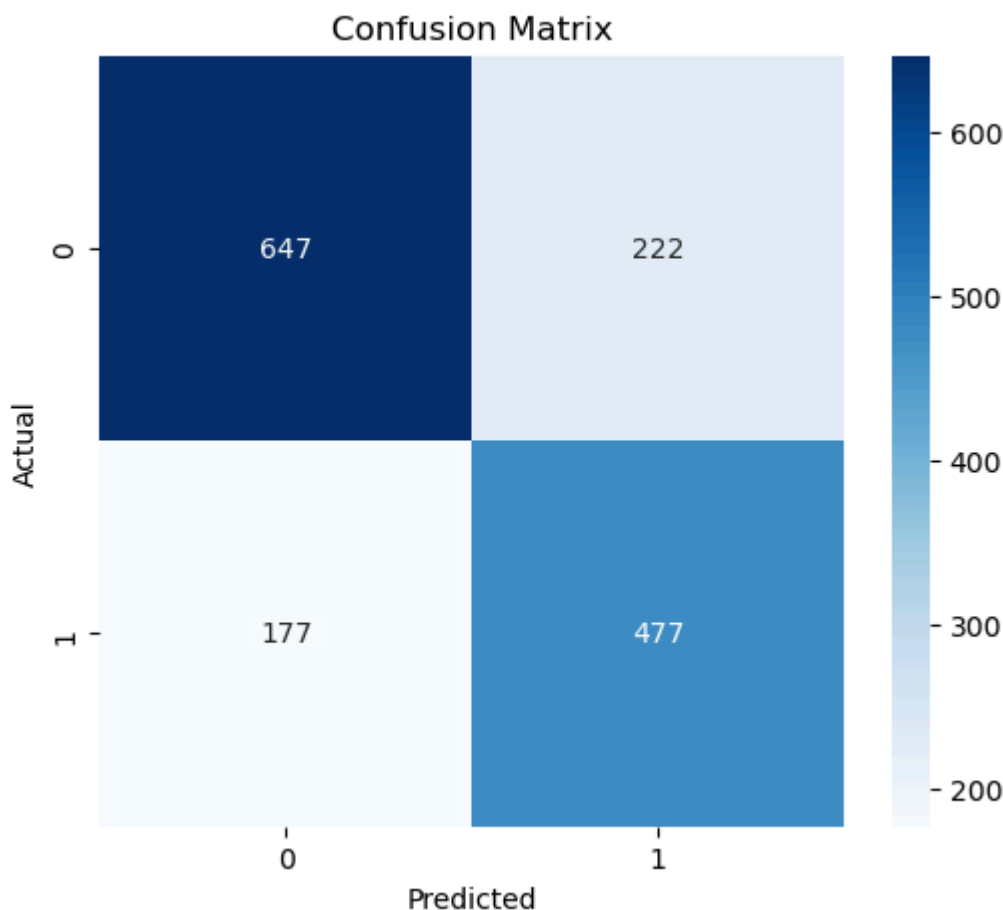
```
In [86]: from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

sns.heatmap(
```

```
cm,  
annot=True,  
fmt='d',  
cmap='Blues'  
)  
  
plt.title('Confusion Matrix')  
plt.xlabel('Predicted')  
plt.ylabel('Actual')  
  
plt.show()
```



Confusion Matrix Analysis

The confusion matrix provides a detailed view of prediction performance.

True positives and true negatives represent correctly classified tweets, while false positives and false negatives indicate classification errors.

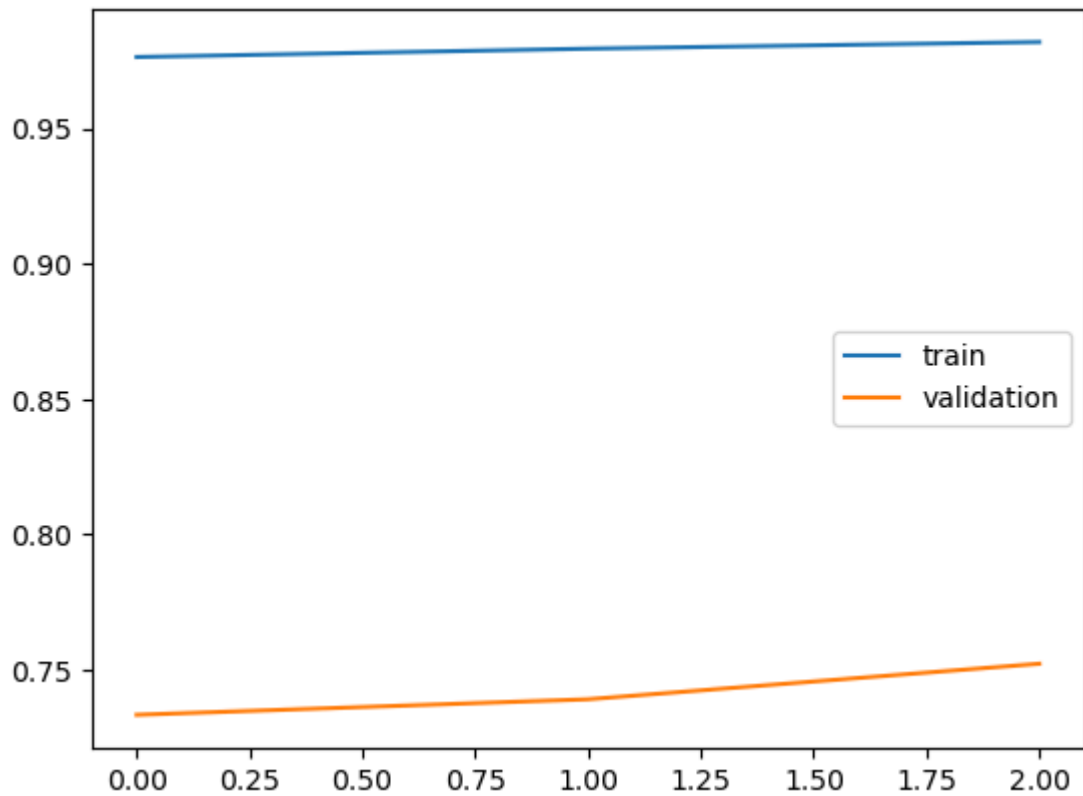
The matrix shows that the model successfully classified the majority of tweets while maintaining balanced performance across both categories.

```
In [87]: history.history.keys()
```

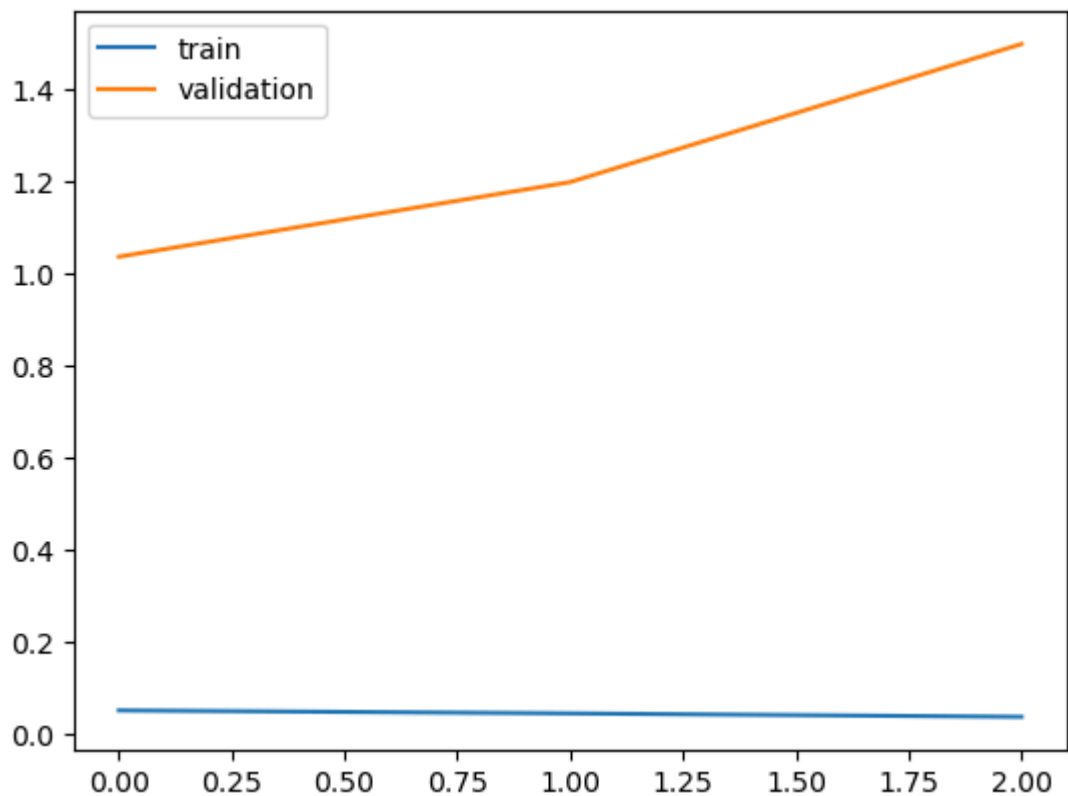
```
Out[87]: dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```

```
In [88]: plt.plot(history.history['accuracy'])  
plt.plot(history.history['val_accuracy'])
```

```
plt.legend(['train', 'validation'])  
plt.show()
```



```
In [89]: plt.plot(history.history['loss'])  
plt.plot(history.history['val_loss'])  
plt.legend(['train', 'validation'])  
plt.show()
```



PREDICTION

```
In [90]: import numpy as np

def predict_tweet(tweet):

    cleaned = clean_text(tweet)

    seq = tokenizer.texts_to_sequences([cleaned])

    padded = pad_sequences(
        seq,
        maxlen=50,
        padding='post'
    )

    pred = model.predict(padded, verbose=0)

    probability = pred[0][0]

    if probability >= 0.5:
        label = "Disaster Tweet"
    else:
        label = "Not Disaster Tweet"

    print("Tweet:", tweet)
    print("Prediction:", label)
    print("Confidence:", round(probability * 100, 2), "%")
```

```
In [91]: predict_tweet(
    "Earthquake destroys multiple buildings in Turkey"
)
```

Tweet: Earthquake destroys multiple buildings in Turkey
Prediction: Disaster Tweet
Confidence: 99.99 %

```
In [92]: predict_tweet(
    "I am dying laughing at this meme"
)
```

Tweet: I am dying laughing at this meme
Prediction: Not Disaster Tweet
Confidence: 1.54 %

```
In [93]: predict_tweet(
    "Flood waters have forced thousands to evacuate"
)
```

Tweet: Flood waters have forced thousands to evacuate
Prediction: Disaster Tweet
Confidence: 99.89 %

```
In [94]: predict_tweet(
    "What a disaster of a football match"
)
```

Tweet: What a disaster of a football match

Prediction: Not Disaster Tweet

Confidence: 15.35 %

Business Impact

This model can assist emergency response teams, government agencies, and social media monitoring systems in identifying disaster-related information quickly. Automated classification can help prioritize critical information during natural disasters and crisis situations.

Conclusion

This project developed a Disaster Tweet Classification system using Natural Language Processing and a Bidirectional LSTM network.

The workflow included text preprocessing, tokenization, sequence padding, deep learning model development, and model evaluation.

The final model achieved approximately 76% accuracy on unseen test data and demonstrated the ability to distinguish real disaster-related tweets from non-disaster tweets.

This project provided practical experience in NLP pipelines, sequence modeling, text classification, and deep learning, while highlighting the importance of preprocessing and contextual understanding in natural language tasks.